

TD n°5 : Diagrammes de séquences

Exercice 1 : *documentation cas d'utilisation*

Montrez, via un diagramme de séquences :

1. les échanges entre un distributeur automatique de billets et une personne porteuse d'une carte bancaire mais qui n'est pas cliente de la banque (uniquement possibilité de retrait).
2. le compléter en mettant en évidence ce que fait le DAB.

Exercice 2 :

Pour chaque situation modélisée précédemment, est-il possible que les instances des classes puissent s'envoyer des messages ? Justifiez :

1. Un répertoire contient des fichiers : *un répertoire peut-il envoyer des messages à ses fichiers et inversement ?*
2. Une pièce contient des murs : *une pièce peut-elle envoyer un message à ses murs et inversement ?*
3. Les modems et les claviers sont des périphériques d'entrée/sortie : *un périphérique d'E/S peut-il envoyer des messages à des modems et des claviers ? un modem peut-il envoyer un message à un clavier et inversement ?*
4. Deux personnes peuvent être mariées : *une personne peut-elle envoyer un message à son conjoint ?*

Exercice 3 :

Proposer le diagramme de séquences pour chacune de ces situations (il est nécessaire d'établir le bout de diagramme de classes associé) :

1. Une banque a notamment des agences et des commerciaux répartis au sein de ces agences. Son chiffre d'affaires est le chiffre d'affaires de toutes ses agences, lui-même se basant sur celui des commerciaux. L'on souhaite pouvoir calculer le CA d'une banque.

2. Pour permettre des achats en ligne, chaque client a un panier électronique qui collectera l'ensemble des articles achetés. Voici les informations du cas d'utilisation *Gestion du panier* :
- a. Le cas d'utilisation se déclenche lorsque le client sélectionne un article présent dans le panier.
 - b. Si le client sélectionne « supprimer l'article », le système le supprime du panier.
 - c. Si le client saisi une nouvelle quantité, le système met à jour la quantité de l'article dans le panier.

Complétez le diagramme de classes, notamment leurs opérations.

Exercice 4 :

Proposer le diagramme de séquences :

```
public abstract class Link()
{
    public abstract void transmit (String m);
}

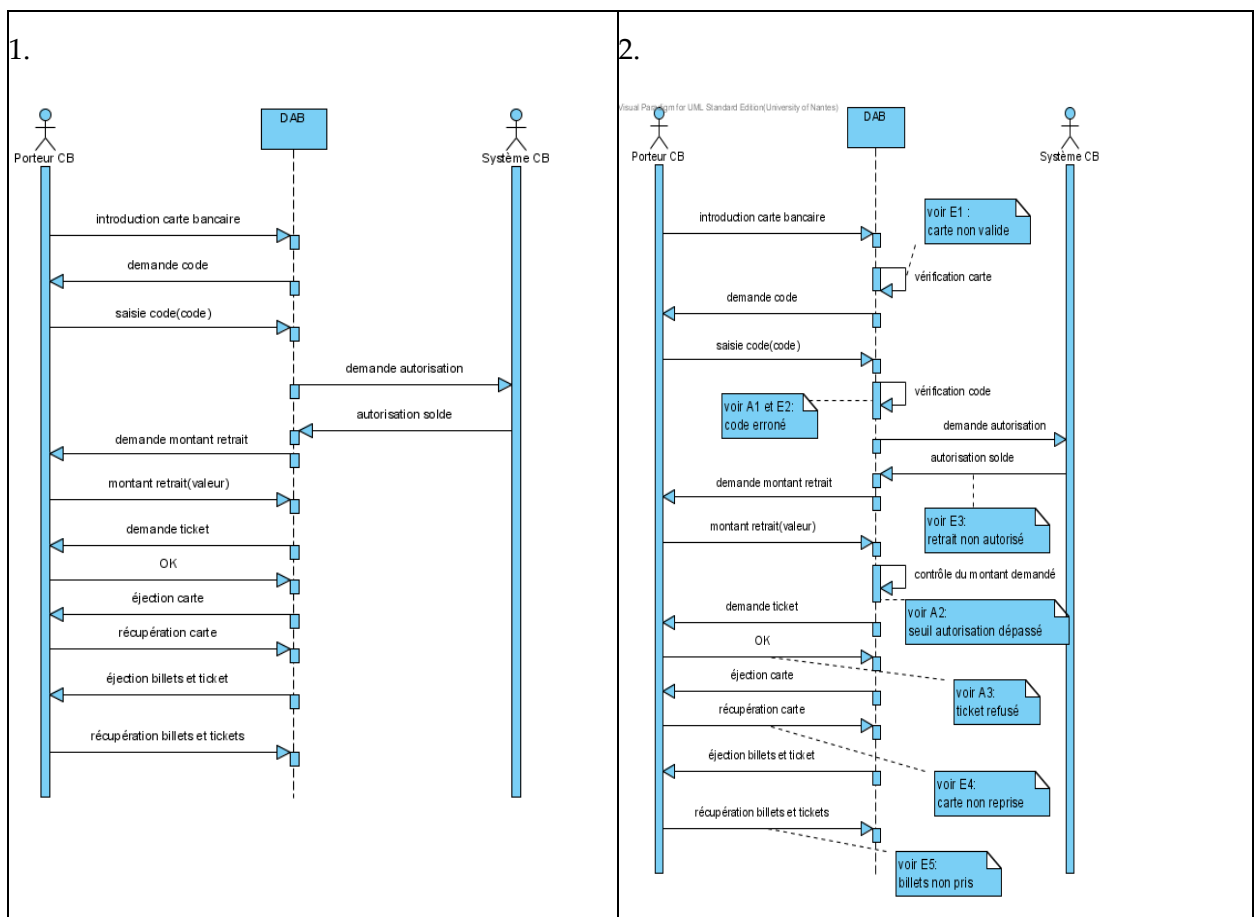
public class NormalLink extends Link {...}
public class PriorityLink extends Link {...}
public class SecureLink extends Link {...}

public class Station
{
    private Link link = new NormalLink();
    private int msg_ind = 0;
    public void sendMessage(String m)
    { link.transmit(msg_ind++ + " " + m);
      if (msg_ind == 10)
        link = new PriorityLink(); }
    }
}
```

Corrigé

Exercice 1 :

Dans la situation 1, des échanges simples. Dans la situation 2, il y a un peu plus d'informations, car on fait ressortir le fait que le DAB fait certains contrôles. Pour ne pas encombrer le message, le renvoi vers d'autres points qui traite des alternatives (A) et des erreurs (E). Si on cherche à mettre tout sur le même diagramme, on sera vite confronté à un diagramme illisible.



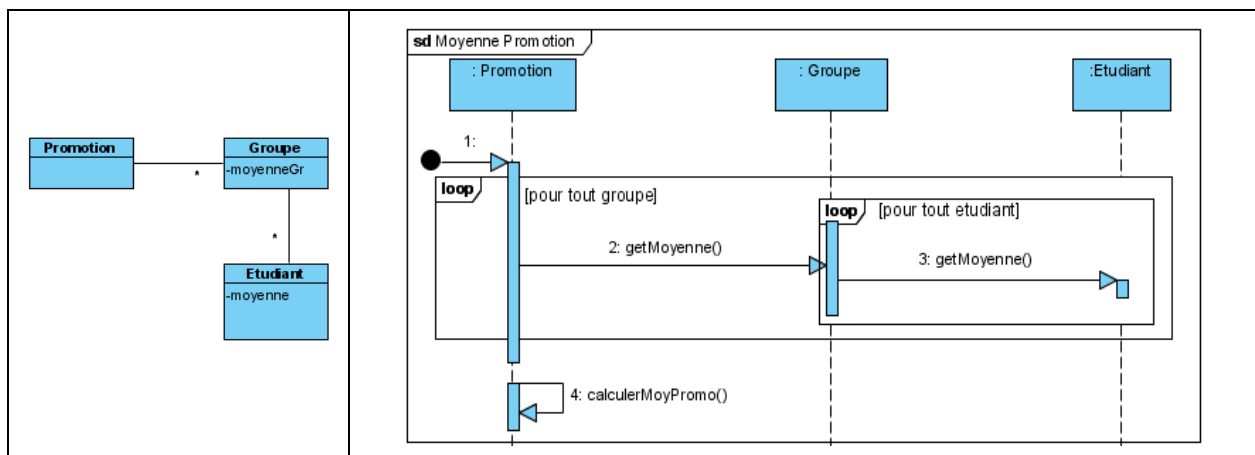
Exercice 2 :

Importance de tenir compte de l'existence d'une association ou d'une dépendance entre deux classes pour qu'elles puissent communiquer :

1. Un répertoire contient des fichiers : *un répertoire peut-il envoyer des messages à ses fichiers et inversement ?*
 - Oui, il y a une association. Aucune info sur la navigabilité => on considère qu'elle est bidirectionnelle. Attirer leur attention que si elle était navigable (par exemple de Répertoire vers Fichier), un fichier ne connaîtrait pas son répertoire, donc pas moyen de lui envoyer un message.
2. Une pièce contient des murs : *une pièce peut-elle envoyer un message à ses murs et inversement ?*
 - Idem que précédemment
3. Les modems et les claviers sont des périphériques d'entrée/sortie : *un périphérique d'E/S peut-il envoyer des messages à des modems et des claviers ? un modem peut-il envoyer un message à un clavier et inversement ?*
 - Non. Toujours se poser la question : est-ce que deux objets se connaissent (association/dépendance) ? Pas le cas ici.
4. Deux personnes peuvent être mariées : *une personne peut-elle envoyer un message à son conjoint ?*
 - Oui. Association.

Exercice 3 :

1. Le même principe que le diagramme de séquences de la moyenne de la promotion vu en cours dont voici le diagramme :

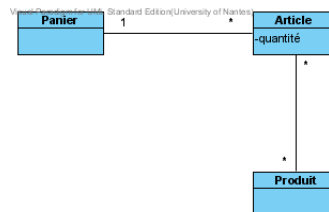


Leur faire faire 2 solutions : une en contrôle centralisé, donc tout part de Banque et une en contrôle décentralisé, donc Banque et Agence jouent toutes deux un rôle dans le diagramme de séquences => Principe de Délégation !!

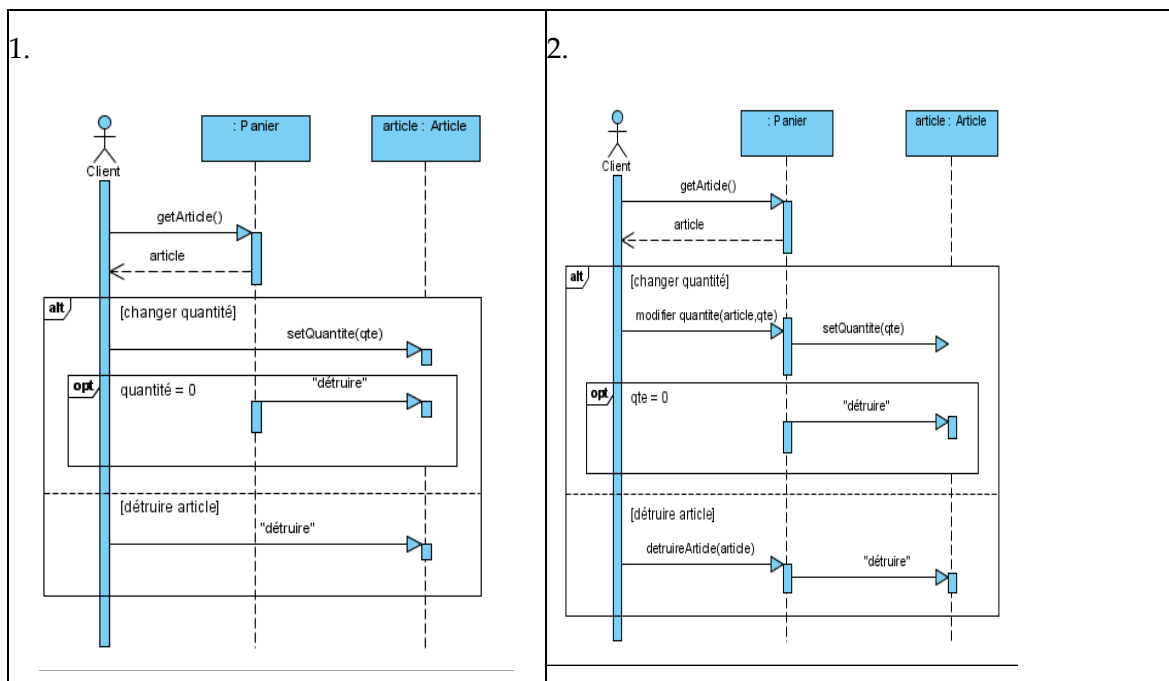
Le dernier appel (le n° 4) n'est pas très correct. Faites-les réfléchir dessus !

2.

- a. diagramme de classes : (une autre version en mettant Article en classe-association est également adaptée)



- b. diagramme de séquences : pour le diagramme de séquences, je vous propose deux solutions : la 1°, les opérations sont directement déclenchées sur Article, alors que dans la seconde, on passe par Panier. Ils doivent donc impacter le diagramme de classes en conséquent. Dans la solution 1, pas de changements particuliers (get et set existent) alors que dans la solution 2, il faut ajouter les deux opérations dans Panier (*modifierQuantite(article,qte)* et *destruireArticle(article)*) qui elles-même iraient déclencher le set et les opérations de destructions.

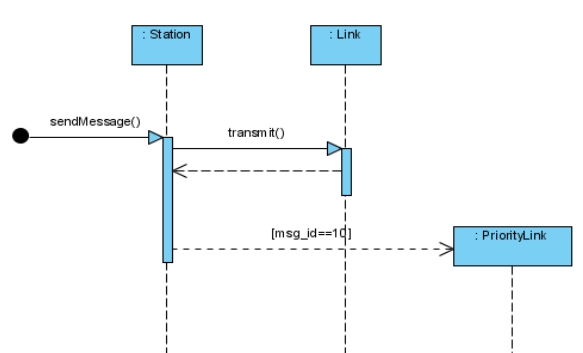


Pour la mise à jour du diagramme de classes, ils peuvent mettre ces get et set pour les besoins de l'exo, mais en cours je leur ai dit qu'on ne mettait pas les get et les set car c'est évident qu'il y en a.

Merci de bien attirer leur attention qu'on modifie un article précis, d'où le 'article :Article' dans les diagrammes de séquences, qui plus est, est passé en paramètre à chaque fois.

Il serait intéressant de déduire ces classes en Java, en respectant pleinement le diagrammes de classes et le diagramme de séquences (deux versions selon diagramme de séquences 1 ou diagramme de séquences 2)

Exercice 4 :



Il faut attirer leur attention qu'il est plus aisé (et surtout plus rapide) de comprendre ce qui se passe en regardant le diagramme de séquences qu'en lisant le code.

Vous remarquez que depuis une instance de la classe Station, il y a un envoi de message vers une instance de Link, alors que cette dernière est abstraite. Cependant, dans la classe Station, l'instance link est créée à partir de NormalLink mais elle est déclarée comme étant de type Link, ce qui permet de lui affecter une autre instance d'une classe sous-classe de Link. L'idée est donc de voir ce que les étudiants vont proposer comme envoi de message "transmit" depuis Station vers quelle destination ... et d'en discuter avec eux.